Presentation Notes — Cool Roofing Cost & Impact Predictor

Smart India Hackathon 2024 *Slide-by-slide speaker guide with all values*

SLIDE 1 — Title Slide

Opening line: > Good morning / Good afternoon, respected judges and fellow participants.

We are presenting our solution for Smart India Hackathon 2024 — **Cool Roofing Cost and Impact Predictor** — under the Smart Infrastructure and Green Buildings track.

What to say:

We built a complete end-to-end machine learning system that answers five critical questions about a cool roofing installation — before a single tile is placed.

Our system takes **9 building parameters** and produces **5 predictions** in under a second.

The tech stack is: Python for all ML work, XGBoost and Random Forest as our models, Streamlit for the web application, and Plotly for interactive visualisations.

Point to the 4 stat callouts: - 2,500 training records in our dataset - 9 input features — all available before installation - 5 trained ML models covering 3 regression and 2 classification tasks - 98.9% cost prediction accuracy ($R^2 = 0.9843$ on held-out test set)

Let me walk you through the problem and our solution.

SLIDE 2 — Problem Statement

What to say:

Let me paint the picture of the gap we are filling.

In India, cool roofing is growing rapidly as a solution to urban heat islands and rising electricity bills. But there is a critical blind spot: **no data-driven tool exists** that tells a building owner or contractor what to expect before installation.

Four specific pain points — point to each card:

1. No Cost Forecast: Building owners commit anywhere from ₹1 lakh to ₹18 lakh — a massive range — with no reliable estimate. Contractors quote entirely from personal experience. This leads to disputes, abandoned projects, and no competitive pressure to be accurate.

2. Energy Uncertainty: The whole value proposition of cool roofing is reducing electricity bills. But nobody knows the actual monthly savings until 3 to 6 months after installation. Owners take it on faith.

3. No Temperature Reduction Estimate: What temperature drop will this building see? Currently zero pre-installation predictability. A building owner cannot compare contractors on this metric.

4. Configuration Guessing: Five cool roofing methods exist — basic coating, tiles, sheet with coating, tiles with coating, reflective coating. Three quality tiers — basic, intermediate, high. That is 15 combinations. Contractors pick based on gut feeling or profit margin, not on which combination gives the best outcome for this specific building.

We solved all four with one unified ML system.

SLIDE 3 — Proposed Solution

What to say:

Our proposed solution is a **5-model ML ensemble** that delivers all predictions from 9 pre-installation building inputs.

Walk through the 5 output cards (left to right):

1. **Installation Cost** — XGBoost regression. $R^2 = 0.984$ on held-out test set. MAE = ₹69,426 — average prediction error across all test buildings.
2. **Temperature Drop** — Random Forest regression. $R^2 = 0.935$. Average error just 0.463°C — predicts post-install roof temperature to within half a degree.
3. **Units Saved per Month** — Random Forest regression. $R^2 = 0.944$. Average error 5.41 kWh/month — equivalent to ₹38 uncertainty at ₹7/kWh tariff.
4. **High Rating ≥4 Stars** — Random Forest classifier. 65.5% accuracy, AUC = 0.728. Baseline was 56.1%.
5. **Customer Referral** — Random Forest classifier. 62.0% accuracy, F1 = 0.505. Baseline was 57.1%.

Point to the 9-input section: The inputs are: building_size, building_age, num_ac_units, avg_roof_temp_before, avg_units_before, location_type, roofing_type, cool_roofing_method, quality. All 9 are available before installation begins. No post-install data used.

Concrete example (say from memory): A 900 sq ft urban building, age 11 years, 1 AC unit, concrete slanted roof, tiles method, basic quality — - Our XGBoost model predicted ₹3,10,742 installation cost - The actual cost was ₹3,13,700 - That is **0.94% prediction error** — under 1%

SLIDE 4 — EDA: Target Distributions

What to say:

These three histograms come directly from our Jupyter notebook. They show the raw distributions of what we are predicting.

Left — Installation Cost histogram: Right-skewed distribution. Average cost is ₹10.9 lakhs, but the range spans ₹55,300 at the low end (small basic rural building) to ₹50.5 lakhs at the high end (large high-quality urban installation). The orange vertical line marks the mean.

This right skew is why we chose XGBoost over linear regression — linear models struggle with heavy-tailed targets.

Centre — Temperature Drop histogram: Near-normal distribution centred around 8.1°C. Most buildings see a 5 to 12°C reduction in roof temperature. The clean bell shape tells us this is a stable, learnable signal. Random Forest regression is appropriate here.

Right — Units Saved per Month histogram: Right-skewed again. Average 46 kWh per month — about ₹320 savings per month at ₹7/kWh tariff. Buildings with very high baseline usage (factories, large commercial) save the most.

Point to the 4 stat callouts below the chart: - Average cost: ₹10.9L - Average temperature drop: 8.1°C - Average monthly units saved: 46 kWh - Dataset rows after cleaning: 2,472

SLIDE 5 — EDA: Categorical & Feature Analysis

What to say:

Top row — categorical balance check:

Left chart — Existing Roof Type: The dataset is balanced across concrete_flat, concrete_slanted, tiles, and sheet roof types. No category is so rare that the model fails to learn its pattern.

Right chart — Cool Roofing Method: Five methods in the dataset, each with 400–550 examples. Again well-balanced — our models see enough of each method.

Bottom row — cost segmentation (this is where the interesting signal is):

Left chart — Average Cost by Location Type: - Dense urban: ~₹14L average - Urban: ~₹11L average - Intermediate: ~₹9L average - Rural: ~₹7L average

A 2× cost difference between dense urban and rural. This drives `location_type` to appear in the top 5 features for the cost model.

Right chart — Average Cost by Quality Tier: - Basic: ~₹5L average - Intermediate: ~₹11L average - High: ~₹18L average

High quality is 3.6× more expensive than basic. **Quality is the single strongest cost predictor.** The feature importance chart on the next slide confirms this directly.

SLIDE 6 — ML Pipeline Architecture

What to say:

Walk through each stage left to right:

Stage 1 — Raw Data: We start with 2,500 building records in `cool_roofing_data.csv`. The dataset has 15 columns — 9 that we use as input features, plus 6 target-related columns. We found 106 missing values across 4 columns, and 32 rows where temperature before installation was logged as 120°C — physically impossible for any rooftop in India. These are sensor errors or placeholder values.

Stage 2 — Clean and Transform: The 120°C values are replaced with NaN rather than deleting those rows — this preserves them for cost, units, and rating models that don't need temperature. Missing numeric values are filled with training-set medians. Four categorical columns are one-hot encoded using `pd.get_dummies()`, expanding the feature space from 9 inputs to 22 model features.

Stage 3 — 5 ML Tasks: We define 3 regression targets and 2 classification targets. Dataset split is 70% train (1,730 rows), 15% validation (371 rows), 15% test (371 rows). `random_state=42` ensures reproducibility.

Stage 4 — Train and Tune: RandomizedSearchCV with 40 iterations and 5-fold cross-validation finds near-optimal hyperparameters for each model. Models are saved as `.joblib` files — loading takes 50ms vs 2 seconds for retraining.

Algorithm summary (bottom strip): - Cost → XGBoost (n_estimators=2000, depth=5, lr=0.06, λ=0.5) - All others → Random Forest (n_estimators=300, depth=12, class_weight=balanced)

Stage 5 — Streamlit App: 5-page web application. Home, Dashboard (8 charts), Predict (single building), Batch CSV upload, and Recommender (ranks 15 method×quality combos).

SLIDE 7 — Regression Results & Feature Importance

What to say:

Top chart — Model R^2 vs Baseline R^2:

Three grouped bars, one pair per regression task.

- **Installation Cost (XGBoost)**: Model $R^2 = 0.9843$. Baseline $R^2 = 0.0$ (predicting mean). Our model explains 98.4% of cost variance.
- **Temperature Drop (Random Forest)**: Model $R^2 = 0.935$. Near-zero baseline.
- **Units Saved (Random Forest)**: Model $R^2 = 0.944$.

The baseline bars are intentionally tiny — the visual contrast makes the improvement obvious.

Bottom chart — Feature Importances across all 5 models:

Five panels, one per model. Key observations:

- **Cost model**: `quality_high` and `building_size` dominate. Makes physical sense — high quality triples the cost, and cost scales with area.
- **Temperature drop**: `avg_roof_temperature_before` has importance ~0.40. Physically obvious — a 70°C roof has more room to drop than a 35°C roof.
- **Units saved**: `avg_units_before` has importance ~0.44. More baseline usage = more opportunity to save. The model has correctly identified that savings are proportional to baseline consumption.
- **High rating**: `avg_units_before` and `num_ac_units` are top features — buildings where cool roofing makes a visible electricity difference produce happier customers.
- **Referral**: Similar to rating — customers who see measurable benefit are more likely to recommend.

If any of these importances had been dominated by irrelevant features, we would suspect a data leakage bug. The sensible top features are a validation that our pipeline is clean.

SLIDE 8 — Cost Model Deep Dive

What to say:

Scatter plot (left): Installation cost (₹ Lakhs) on the Y-axis, building size on the X-axis, each dot coloured by quality tier.

The three colour bands are clearly separated — green (basic) at the bottom, orange (intermediate) in the middle, navy (high) at the top. Within each band, size adds a second dimension of variation. A 3,000 sq ft high-quality installation (top right) costs roughly ₹40–50L. A 500 sq ft basic installation (bottom left) costs ₹1–3L.

This non-linear, multi-variable interaction is exactly why XGBoost outperforms linear regression here. A linear model would draw one straight line; XGBoost captures the separate curves within each quality band.

Metrics panel (top right): - R^2 (Test) = 0.9843 — the model explains 98.4% of test set cost variance - MAE = ₹69,426 — average absolute prediction error - RMSE = ₹1,11,562 — root mean squared error (penalises large errors) - Baseline MAE = ₹6,08,878 — what you get predicting the mean for every building - Our model is **8.8× better than baseline** on MAE

Worked example (bottom right): - Building: 900 sq ft, urban, age 11, 1 AC unit, concrete slanted, tiles method, basic quality - Predicted: **₹3,10,742** - Actual: **₹3,13,700** - Error: **0.94%** — under 1%

How XGBoost works (bottom strip): 2,000 trees built sequentially. Tree 1 starts from the dataset mean (₹6.8L average). Each subsequent tree corrects the residuals — the errors — of all previous trees. Learning rate 0.06 ensures no single tree overcorrects. After 2,000 trees, the cumulative correction brings the prediction from ₹6.8L down to ₹3.1L for this specific building. L2 regularisation (λ=0.5) prevents any individual leaf weight from growing too large.

SLIDE 9 — Classification Results

What to say:

Two confusion matrices from our notebook — one for each classification task.

Left matrix — High Rating (≥4 stars), Blues colormap:

Read the matrix as: - Top-left (True Negatives): Buildings that did NOT get high ratings and the model correctly predicted NOT high - Top-right (False Positives): Buildings the model predicted high but got low rating — false alarm - Bottom-left (False Negatives): Buildings that DID get high ratings but model predicted low — missed - Bottom-right (True Positives): Correctly predicted high rating

Results: - Accuracy: **65.5%** vs stratified baseline of 56.1% - ROC-AUC: **0.728** — above 0.5 (random), well below 1.0 (perfect). Meaningful discriminative ability. - F1: **0.63**

Model: Random Forest with `class_weight='balanced'` to handle the 57:43 class split.

Right matrix — Customer Referral, Greens colormap: - Accuracy: **62.0%** vs baseline 57.1% - F1: **0.505** - ROC-AUC: **0.646**

Why is referral accuracy lower than rating? Referral depends on social factors — personal network, neighbourhood norms, timing — that are not in the dataset. Physical building features predict referral imperfectly by design. The model still performs better than chance, and the referral probability is used as a 10% component of the recommender score.

Why class_weight='balanced'? Without it, the Random Forest would predict "not high rating" for every building and still get 56.1% accuracy — matching the majority class. Balanced weighting forces the model to actually learn what separates satisfied from unsatisfied customers.

SLIDE 10 — Customer Satisfaction Analysis

What to say:

Left chart — Rating Distribution bar chart:

Five bars coloured by satisfaction level (red to green): - 1 star: approximately 310 buildings - 2 stars: approximately 490 buildings - 3 stars: approximately 715 buildings — this is the mode - 4 stars: approximately 580 buildings - 5 stars: approximately 380 buildings

65% of buildings received 4 or 5 stars. This is the source of our binary classification threshold — ≥4 stars = high rating = positive outcome.

The 57:43 split (not-high : high) is a mild class imbalance — manageable with `class_weight='balanced'` rather than oversampling techniques.

Right chart — Referral Pie: Approximately 64% No Referral, 36% Referral. A 64:36 split — more imbalanced than rating. This is why referral classification is harder and achieves lower accuracy.

Three insight cards below:

1. **65% High Rating**: Majority of customers are satisfied. The cool roofing installation genuinely delivers value.
2. **~36% Referral Rate**: More than 1 in 3 customers refer the contractor — strong word-of-mouth signal. Our model predicts this probability and uses it in the recommender.
3. **Class Imbalance Handled**: Without `class_weight='balanced'`, Random Forest would achieve 57% accuracy by predicting "no referral" every time. Balanced weights bring the model to 62% with meaningful F1=0.505 — the model is actually learning to identify potential referrers.

SLIDE 11 — Live Application

What to say:

We built a 5-page Streamlit application that puts all 5 models into one tool. Let me walk through each page.

Page 1 — Home: The landing page shows live KPIs computed at startup: 2,472 dataset records, ₹10.9L average installation cost, 8.1°C average temperature drop, 46 kWh average monthly units saved. Green indicators confirm each of the 5 models loaded successfully from disk.

Page 2 — Dashboard: 8 interactive Plotly charts. Four sidebar filters: location type, quality tier, cool roofing method, and building size range. All 8 charts respond simultaneously when a filter changes.

Charts include: cost histogram (by quality), method × quality heatmap (average cost grid), cost vs size scatter (bubble = AC units), temperature drop boxplot (by method), units saved scatter (vs baseline usage), rating distribution bar, referral rate bar, and correlation heatmap.

Example: Set filter to urban + high quality. Average cost chart updates to show the ₹14L+ segment. Heatmap highlights tiles_with_coating + high as most expensive.

Page 3 — Predict: User enters 9 features using sliders and dropdowns. Instantly gets 5 model predictions plus a financial estimate: monthly savings = units_saved × tariff (₹/kWh), annual savings = monthly × 12, payback period = cost / annual_savings.

Using our row 22 example: 900 sqft urban tiles basic → ₹3,10,742 cost, 8.2 kWh saved at this configuration, ₹68/month savings at ₹7/kWh tariff.

Page 4 — Batch Predict: Upload any CSV file. The system auto-maps column aliases — if the CSV has a column called "area" it maps to `building_size`. Missing values filled with training-set medians (not batch medians — no leakage). 2,500 rows predicted in under 0.3 seconds using vectorised pandas operations — no Python loops.

Page 5 — Recommender: User enters 7 fixed features (not method or quality — those are what we optimise over). The system runs all 5 models for all 15 method × quality combinations, computes a weighted score for each, and produces a ranked table.

Weights: cost (−0.25, lower is better), temperature drop (+0.25), units saved (+0.25), high rating probability (+0.15), referral probability (+0.10). All values normalised 0–1 before weighting.

Example: Urban 900 sq ft building → tiles_with_coating + high quality = top weighted score 0.7842.

SLIDE 12 — Future Scope & Impact

What to say:

Four development phases (walk through the cards):

REST API Layer (top left): Wrap all 5 models in a FastAPI backend. Currently the app requires a Python environment. An API layer enables mobile apps and third-party contractor platforms to call predictions with a simple HTTP POST request. Timeline: Q1 2025.

User Auth & Logs (top right): Add PostgreSQL to store every prediction — inputs, outputs, timestamp, contractor ID. This enables audit trails (useful for disputes), usage analytics, and drift detection. If predictions start diverging from actuals 6 months later, the logs catch it automatically.

Live Retraining (bottom left): As new installation records come in from completed projects, automatically retrain all 5 models on a nightly schedule. The system improves continuously — the more installations, the better it predicts.

IoT Temperature Sensors (bottom right): Partner with hardware providers to deploy sensors on rooftops before and after installation. Automatic label generation eliminates manual data entry — the training dataset grows without human effort.

Impact Summary (bottom section — read each bullet directly):

- **98.4% cost prediction accuracy** — our XGBoost model achieves $R^2 = 0.9843$ and MAE = ₹69,426 on a held-out test set of 371 buildings. Contractors can quote within ₹70K of actual cost before any site visit.
- **Pre-install energy forecast** — owners can calculate payback period before signing. At ₹7/kWh tariff, 46 kWh/month savings = ₹3,864/year. A ₹10L installation pays back in 2.6 years.
- **2,500 rows in 0.3 seconds** — vectorised batch processing using pandas + numpy + joblib. No Python loops. Tested and confirmed.

Closing: > Thank you. We are happy to take questions or do a live demo of the Streamlit application.

Term-by-Term Glossary

Plain-language definitions + the specific numbers from our project

---

R^2 — R-Squared (Coefficient of Determination)

What it is: A number between 0 and 1 that tells you how well your model's predictions match reality.

- $R^2 = 0$ → model is no better than guessing the average every time
- $R^2 = 1$ → model predicts every value perfectly
- $R^2 = 0.98$ → model explains 98% of the variation in the data; only 2% is unexplained

How to think about it: Imagine 371 test buildings, each with a real installation cost. Instead of predicting, a lazy "baseline" model just says "₹10.9 lakhs" for every single building. That baseline $R^2 \approx 0.0$. Our XGBoost model's $R^2 = 0.9843$ means it is explaining 98.43% of what makes one building cost ₹2L and another cost ₹40L.

Our numbers: | Model | R^2 Score | Interpretation | |—|—|—| | Installation Cost (XGBoost) | **0.9843** | 98.4% of cost variation explained | | Temperature Drop (RF) | **0.9350** | 93.5% of temp drop variation explained | | Units Saved (RF) | **0.9440** | 94.4% of savings variation explained |

When judges ask: "Is 0.98 good?" — Yes, exceptionally. Anything above 0.90 for a real-world regression task with noisy human data is strong. 0.98 is publication-quality.

XGBoost — eXtreme Gradient Boosting

What it is: A machine learning algorithm that builds many decision trees one by one, where each new tree corrects the mistakes of all previous trees. "Boosting" = sequential improvement. "Gradient" = it uses the math of gradients (slopes) to decide how to correct.

Simple analogy: You have a bad guesser (Tree 1) that says ₹6.8L for every building. Its errors are: Building A was ₹3.1L (off by ₹3.7L), Building B was ₹40L (off by ₹33.2L). Tree 2 learns from these errors and corrects them. Tree 3 corrects Tree 2's residual errors. After 2,000 trees, the combined prediction is very precise.

Why we chose XGBoost for cost: - Cost distribution is heavily right-skewed (₹55K to ₹50L range) - Non-linear interaction between quality, size, and location - XGBoost achieved $R^2 = 0.9843$ vs Random Forest's 0.9797 on cost — a clear winner - Built-in L2 regularisation (λ parameter) prevents overfitting

Our hyperparameters (and why): | Parameter | Value | Why | |—|—|—| | n_estimators | 2000 | 2,000 trees; more trees = finer corrections | | max_depth | 5 | Each tree can branch at most 5 levels deep; prevents over-complex trees | | learning_rate | 0.06 | Each tree contributes 6% of its correction; small = stable, no overcorrecting | | λ (L2 reg) | 0.5 | Penalises large leaf weights; reduces variance | | subsample | 0.9 | Each tree trained on 90% of data (random sample); adds diversity |

Random Forest

What it is: Builds hundreds of decision trees independently (not sequentially like XGBoost), each trained on a random subset of data and random subset of features. Final prediction = average of all tree predictions (regression) or majority vote (classification). "Forest" = many trees. "Random" = randomness prevents all trees from being identical.

Why we used RF for temperature drop, units saved, rating, and referral: - These targets are smoother and less skewed than cost - Random Forest handles classification (high rating, referral) natively with `class_weight` option - Less hyperparameter-sensitive than XGBoost — faster to tune - Performs within 0.5% of XGBoost on all non-cost tasks

Our hyperparameters: | Parameter | Value | Why | |—|—|—| | n_estimators | 300 | 300 independent trees; enough diversity without over-computing | | max_depth | 12 | Deeper trees = captures complex

patterns; regularised by the forest average | | class_weight | balanced | Upweights minority class (referral=yes) so model doesn't just predict majority |

---

70 / 15 / 15 Split

What it is: Dividing the dataset into three non-overlapping subsets before any training begins: - **Training set (70%):** the data the model learns from - **Validation set (15%):** used during training to tune hyperparameters; model never trained on it - **Test set (15%):** held out completely until final evaluation; gives an unbiased performance estimate

Our numbers: | Split | Rows | Purpose | |—|—|—| | Train | 1,730 (70%) | Model learns from these | | Validation | 371 (15%) | RandomizedSearchCV cross-validates here | | Test | 371 (15%) | Final R^2, MAE, RMSE reported from these |

Why 70/15/15 and not 80/20: 2,500 rows is a medium dataset. With 80/10/10, the test set would have only 250 rows — too small to get reliable metric estimates. 371 test rows gives stable percentile-based error estimates. The validation set needs 371 rows too — RandomizedSearchCV runs 40 × 5-fold = 200 model fits on it.

Why `random_state=42`: Setting a fixed seed ensures the same 1,730/371/371 rows land in the same splits every time the code runs. Without it, two runs of the same notebook produce different test sets and therefore different R^2 values — making results non-reproducible.

MAE — Mean Absolute Error

What it is: Average of the absolute differences between predicted and actual values. Every error counts equally regardless of size.

Formula: `MAE = average(|predicted - actual|)` for all test buildings

Our numbers: - Cost MAE = **₹69,426** — average prediction error across 371 test buildings - Baseline MAE = **₹6,08,878** — if you just predict ₹10.9L for every building - **Our model is 8.8× better than baseline on MAE**

Plain English: On average, our predicted installation cost is ₹69,426 away from the real cost. For a ₹10.9L average installation, that is a 0.64% relative error.

RMSE — Root Mean Squared Error

What it is: Like MAE but penalises large errors more heavily. Squaring the errors before averaging means a ₹5L error counts 100× more than a ₹50K error.

Formula: `RMSE = sqrt(average((predicted - actual)²))`

Our numbers: - Cost RMSE = **₹1,11,562** - RMSE > MAE always (₹1.1L > ₹69K) — this tells us a few buildings have large errors even though the average is small

Why report both: MAE tells you typical error. RMSE tells you worst-case tendency. If RMSE >> MAE, you have outlier predictions. Our ₹1.1L RMSE vs ₹69K MAE is a 1.6× ratio — acceptable for a heavy-tailed cost distribution.

ROC-AUC — Area Under the Receiver Operating Characteristic Curve

What it is: A metric for classification models. Measures how well the model ranks positives (e.g., will give ≥4 stars) above negatives (will give <4 stars), across all possible decision thresholds.

- AUC = 0.5 → model is random (coin flip)
- AUC = 1.0 → model perfectly separates positives from negatives
- AUC = 0.728 → 72.8% of the time, the model correctly ranks a high-rater above a low-rater

Our numbers: | Task | AUC | Meaning | |—|—|—| | High Rating | **0.728** | Model has real discriminative ability | | Customer Referral | **0.646** | Weaker but above chance; social factors limit the ceiling |

Why not just use accuracy: Accuracy can be misleading. If 57% of buildings are "not high rating," a model that always predicts "not high rating" gets 57% accuracy while learning nothing. AUC sees through this by evaluating ranking ability.

F1 Score

What it is: A single number combining Precision and Recall for a classifier.

- **Precision:** Of all buildings the model predicted as "high rating" — what fraction actually were?
- **Recall:** Of all actual "high rating" buildings — what fraction did the model catch?
- **F1 = 2 × (Precision × Recall) / (Precision + Recall)**

Our numbers: | Task | F1 | Interpretation | |—|—|—| | High Rating | **0.63** | Reasonable balance between catching true high-raters and avoiding false alarms | | Referral | **0.505** | Near-0.5 = harder task; model still better than baseline |

F1 = 0.505 doesn't mean the model is bad. It means: for a social behaviour (referral) that our 9 physical features only partially explain, the model still extracts meaningful signal.

Baseline Model (DummyRegressor / DummyClassifier)

What it is: The stupidest possible model — it ignores all input features and always predicts the training set mean (for regression) or the majority class (for classification). Used as the lower bound to prove your model actually learned something.

Our baselines: | Task | Baseline Value | Our Model | |---|---|---| | Cost MAE | ₹6,08,878 | ₹69,426 (8.8× better) | | Cost R^2 | 0.0 | 0.9843 | | High Rating Accuracy | 56.1% | 65.5% | | Referral Accuracy | 57.1% | 62.0% |

Why it matters for judges: Every model must beat its baseline or it has failed to learn anything. All 5 of our models significantly beat their respective baselines.

One-Hot Encoding (`pd.get_dummies`)

What it is: Converting categorical text columns into binary (0/1) numeric columns so machine learning algorithms can process them. Each unique category value becomes its own column.

Example — `location_type` with values: urban, rural, intermediate, dense_urban:

```
Before encoding:      After encoding:
location_type         location_type_dense_urban  location_type_intermediate
location_type_rural  location_type_urban
urban          →      0                          0
0                    1
rural          →      0                          0
1                    0
```

Our numbers: - Input: 9 features (4 are categorical: `location_type`, `roofing_type`, `cool_roofing_method`, `quality`) - After `pd.get_dummies`: 22 model features (9 numeric + 13 binary columns from 4 categorical)

Why `drop_first=False`: We keep all categories as columns. This preserves full interpretability — each category has its own feature importance score. `drop_first=True` removes one reference category but doesn't improve model performance here.

Feature Importance

What it is: A score per input feature that measures how much that feature reduced prediction error across all trees in the forest (or boosted trees in XGBoost). Higher = that feature contributed more to good predictions.

Our key findings: | Model | Top Feature | Importance | Why | |---|---|---|---| | Cost (XGBoost) | `quality_high` | ~0.38 | High quality triples cost | | Cost (XGBoost) | `building_size` | ~0.22 | Cost

scales with area | | Temp Drop (RF) | `avg_roof_temperature_before` | ~0.40 | Higher starting temp = bigger potential drop | | Units Saved (RF) | `avg_units_before` | ~0.44 | Baseline usage determines savings ceiling | | High Rating (RF) | `avg_units_before` | ~0.18 | Visible electricity savings drive satisfaction |

Sanity check: If irrelevant features (e.g., random noise) had topped the importance charts, that would signal data leakage or a pipeline bug. All top features make physical or economic sense — a sign the pipeline is clean.

Cross-Validation (5-Fold CV)

What it is: During hyperparameter tuning, the validation set is split into 5 equal folds. The model trains on 4 folds and validates on the 5th, cycling through all 5 folds. The 5 validation scores are averaged. This gives a more stable estimate of performance than a single train/validate split.

Our usage: RandomizedSearchCV runs 40 random hyperparameter combinations × 5 folds = 200 model fits per task. The combination with the best average cross-validation R^2 (or F1 for classifiers) wins.

Why 5-fold: Standard practice. More folds = more stable estimate but slower. 5-fold balances reliability and compute time for a 1,730-row training set.

RandomizedSearchCV (Hyperparameter Tuning)

What it is: Automated search over a range of hyperparameter values. Instead of trying every possible combination (GridSearchCV), it randomly samples 40 combinations and cross-validates each. Much faster when the hyperparameter space is large.

What we tuned (example — XGBoost cost model):

```
param_grid = {
    'n_estimators': [500, 1000, 1500, 2000, 2500],
    'max_depth': [3, 4, 5, 6, 7],
    'learning_rate': [0.01, 0.03, 0.06, 0.1],
    'subsample': [0.7, 0.8, 0.9, 1.0],
    'reg_lambda': [0.1, 0.3, 0.5, 1.0]
}
```

Total combinations: 5 × 5 × 4 × 4 × 4 = 1,600. RandomizedSearchCV evaluated 40 of these. Best found: n_estimators=2000, depth=5, lr=0.06, λ=0.5.

L2 Regularisation (λ — Lambda)

What it is: A penalty added to the loss function during XGBoost training that discourages large leaf weight values. Large leaf weights = model has over-memorised specific training examples. λ=0.5 is a moderate penalty.

Effect of λ on our model: - λ=0 → XGBoost can assign any weight; risk of overfitting - λ=0.5 → Leaf weights are constrained; generalises better to unseen buildings - λ=10 → Over-penalised; model becomes too simple, underfits

Our value: λ=0.5 was chosen by RandomizedSearchCV as the value that maximised validation R^2 on the cost task.

`class_weight='balanced'`

What it is: A parameter for Random Forest (and other sklearn classifiers) that adjusts how much importance the algorithm places on each class during training. The weight for each class is inversely proportional to its frequency.

Formula: `weight(class) = total_samples / (num_classes × samples_in_class)`

Our dataset: - High rating: 43% of 2,472 = ~1,063 buildings - Not high rating: 57% = ~1,409 buildings - Without balanced: model learns to always predict "not high rating" → 57% accuracy - With balanced: minority class (high rating) gets ~1.33× upweight → model learns to distinguish

Result: Accuracy drops from 57% (fake accuracy from always predicting majority) to genuine 65.5% with F1 = 0.63.

Vectorised Batch Inference

What it is: Running predictions on many rows simultaneously using array operations (numpy/pandas), rather than a Python `for` loop over each row.

Python loop (slow):

```
results = []
for row in data.iterrows():  # O(n) Python interpreter overhead
    pred = model.predict([row])
    results.append(pred)
# 2,500 rows: ~4.2 seconds
```

Vectorised (fast):

```
results = model.predict(data)  # One call, internally parallelised
# 2,500 rows: 0.23 seconds — 18× faster
```

Our result: Batch prediction of 2,500 rows in 0.23 seconds, confirmed with `time.time()` benchmarks in the notebook. Feature masking (zeroing excluded columns without retraining) is also vectorised — no additional overhead.

Feature Masking

What it is: Setting unwanted input features to zero rather than removing them from the dataset or retraining a separate model. This lets one trained model handle inputs where some features are missing or excluded.

Why we use it (Recommender page): The Recommender tries all 15 (method × quality) combinations for a fixed building. Instead of training 15 separate models, the single trained model receives the full 22-column feature vector, with `cool_roofing_method_*` and `quality_*` columns swapped to match each combination. Zero means "this category is not selected." Output: 15 predictions in one vectorised call.

Overfitting vs Underfitting

Overfitting: Model memorises the training data including its noise. Performs excellently on training set, poorly on test set. Sign: training R^2 >> test R^2.

Underfitting: Model is too simple. Performs poorly on both training and test sets. Sign: low R^2 everywhere.

Our model: XGBoost cost model: training $R^2 \approx 0.997$, test $R^2 = 0.9843$. A 1.3% gap — slight overfitting but well within acceptable range. L2 regularisation (λ=0.5) and subsample=0.9 control this.

Joblib Model Serialisation

What it is: Saving a trained model to a file (`.joblib`) so it can be loaded later without retraining.

Why it matters: - Retraining XGBoost with 2,000 trees: ~2 seconds per model - Loading saved `.joblib` file: ~50ms - Streamlit app loads all 5 models at startup in ~250ms total

File sizes: | Model | File Size | |—|—| | cost_model_xgb.joblib | ~18 MB | | temp_drop_rf.joblib | ~12 MB | | units_saved_rf.joblib | ~11 MB | | rating_clf.joblib | ~9 MB | | referral_clf.joblib | ~9 MB |

Quick Q&A Cheatsheet

Likely Question	Answer
Why XGBoost for cost but RF for others?	XGBoost achieves R^2 0.9843 vs RF 0.9797 for cost. Sequential boosting handles cost's high-variance right-skewed distribution better.
Why not deep learning?	2,472 rows is too small for neural networks. Ensemble tree methods outperform neural nets on tabular data at this scale.
Why 70/15/15 split?	1,730 training rows gives enough examples per category. 371 test rows gives reliable metric estimates.
What is the baseline?	DummyRegressor predicting mean. MAE for cost = ₹6,08,878 vs our ₹69,426.
Why is temperature drop R^2 lower in final app?	Final app uses production XGBoost model retrained specifically for submission. Notebook uses RF with R^2=0.935. Both are strong.
Why does referral accuracy seem low at 62%?	Referral depends on social factors (network, word of mouth) not in dataset. 62% vs 57% baseline = +5pp improvement. Useful, not definitive.
Can the app handle buildings outside the dataset range?	Predictions degrade outside 300–3,000 sq ft and 1–50 years. A warning is shown on the Predict page.
How fast does the batch predictor run?	2,500 rows in 0.23 seconds on a standard laptop. Vectorised pandas + numpy, no Python loops.